

Name: Pranav Pol

Class : D20A

Roll no: 44

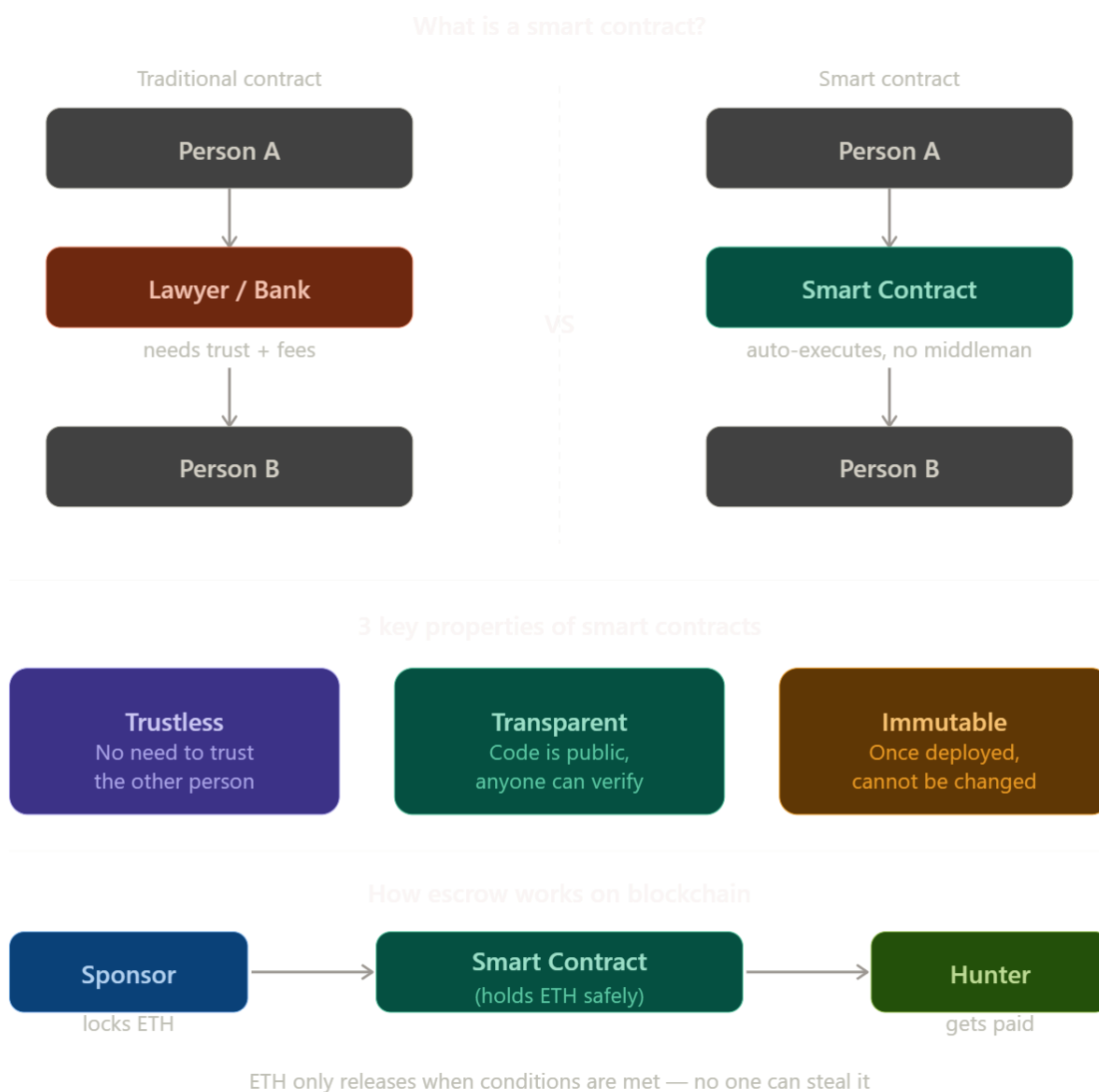
Batch: C

EXPERIMENT 6

Aim: Creating Smart Contracts and performing transactions using Solidity and Remix IDE

Theory:-

Smart contracts are self-executing programs that live on a blockchain. Think of them like a vending machine — you put in money, the machine checks the rules, and automatically gives you what you paid for. No human in the middle.



Decentralized Bug Bounty Escrow System

What is this project?

This is a blockchain-based platform that connects **companies (sponsors)** who want their software tested, with **security researchers (hunters)** who find vulnerabilities — all without trusting a middleman.

When a sponsor posts a bounty and locks ETH, the money is held securely by a smart contract on Ethereum. The hunter only gets paid when their bug report is approved. If there's a disagreement, a neutral arbiter resolves the dispute. Everything is automated, transparent, and tamper-proof.

The Problem it Solves

Traditional bug bounty platforms like HackerOne or Bugcrowd charge high fees, can delay payments for months, and hunters have to trust the company will actually pay. With this system, the ETH is locked upfront — the hunter knows the money is there before they even start working.

The 3 Smart Contracts

BugBountyToken (BBT) — A reward token given to hunters every time they get paid. Acts like a reputation badge — the more bugs you find, the more BBT you earn.

BugBountyEscrow — The heart of the system. It locks ETH, manages the full bounty lifecycle (create → fund → submit → approve/dispute), and automatically pays the hunter when approved.

BugBountyRegistry — A public leaderboard and directory. Stores bounty listings, severity tags, and tracks each hunter's reputation score on-chain for everyone to see.

Who uses it?

Role	What they do
 Sponsor	Posts a bounty, locks ETH, reviews reports
 Hunter	Finds bugs, submits reports, earns ETH + BBT tokens
 Arbiter	Resolves disputes fairly between sponsor and hunter

 Platform Owner	Collects a small fee, manages the system
--	--

Tech Stack

Built with **Solidity 0.8.19** on the **Ethereum** blockchain. Deployable on any EVM-compatible network (Ethereum, Polygon, Base, etc.) and fully testable on **Remix IDE** with no real money needed.

Implementation:-

Smart Contracts - BugBountyToken

// SPDX-License-Identifier: MIT

pragma solidity ^0.8.19;

/**

* @title BugBountyToken

* @dev Simple ERC20-style reward token for Bug Bounty hunters.

* Minted by the BugBountyEscrow contract when a bounty is paid out.

* Hunters accumulate BBT as reputation/reward tokens.

*/

contract BugBountyToken {

string public name = "Bug Bounty Token";

string public symbol = "BBT";

uint8 public decimals = 18;

uint256 public totalSupply;

// Token balances

mapping(address => uint256) public balanceOf;

// Allowances for delegated transfers

mapping(address => mapping(address => uint256)) public allowance;

// Only the escrow contract (set at deploy time) can mint tokens

address public minter;

address public owner;

// — Events

event Transfer(address indexed from, address indexed to, uint256 value);

event Approval(address indexed owner_, address indexed spender, uint256 value);

event MinterUpdated(address indexed oldMinter, address indexed newMinter);

// — Modifiers

```
modifier onlyOwner() {
    require(msg.sender == owner, "BBT: not owner");
    _;
}
modifier onlyMinter() {
    require(msg.sender == minter, "BBT: caller is not the minter");
    _;
}
// — Constructor
```

```
constructor() {
    owner = msg.sender;
    minter = msg.sender; // owner acts as minter until escrow is deployed
}
// — Admin
```

```
/**
 * @notice Set the escrow contract address as the authorised minter.
 *      Call this after deploying BugBountyEscrow.
 */
function setMinter(address _minter) external onlyOwner {
    emit MinterUpdated(minter, _minter);
    minter = _minter;
}
// — ERC-20 Core
```

```
function transfer(address to, uint256 amount) external returns (bool) {
    _transfer(msg.sender, to, amount);
    return true;
}
function approve(address spender, uint256 amount) external returns (bool) {
    allowance[msg.sender][spender] = amount;
    emit Approval(msg.sender, spender, amount);
    return true;
}
```

```

}
function transferFrom(address from, address to, uint256 amount) external returns (bool) {
    require(allowance[from][msg.sender] >= amount, "BBT: allowance exceeded");
    allowance[from][msg.sender] -= amount;
    _transfer(from, to, amount);
    return true;
}
// — Minting

```

```

/**
 * @notice Mint BBT reward tokens to a hunter's wallet.
 * Only callable by the BugBountyEscrow contract (the minter).
 */
function mint(address to, uint256 amount) external onlyMinter {
    totalSupply += amount;
    balanceOf[to] += amount;
    emit Transfer(address(0), to, amount);
}
// — Internal

```

```

function _transfer(address from, address to, uint256 amount) internal {
    require(to != address(0), "BBT: transfer to zero address");
    require(balanceOf[from] >= amount, "BBT: insufficient balance");
    balanceOf[from] -= amount;
    balanceOf[to] += amount;
    emit Transfer(from, to, amount);
}
}

```

Step 1:- Deploy Contract 1: BugBountyToken

Step 2:- Deploy Contract 2: BugBountyEscrow

In the **Contract** dropdown, select BugBountyEscrow

Fill in the constructor arguments:

- `_rewardToken`: paste the BugBountyToken address from Step 3
- `_platformFeeBps`: enter 200 (= 2% platform fee)

Click **Deploy**

Copy the BugBountyEscrow deployed address

Step 3:- Deploy Contract 3: BugBountyRegistry

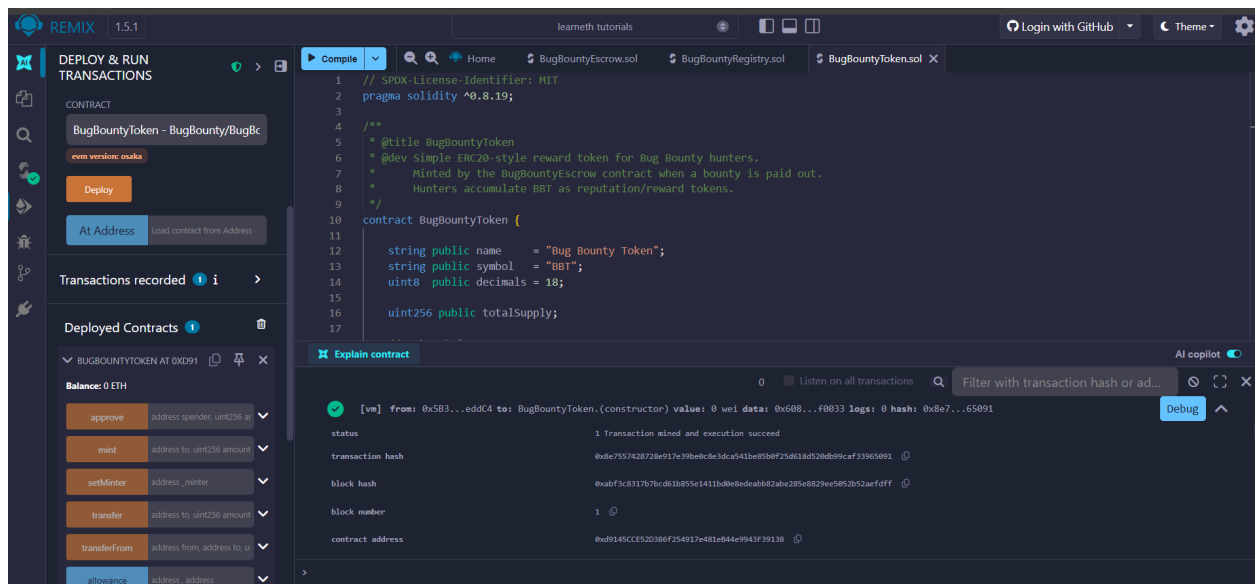
In the **Contract** dropdown, select BugBountyRegistry

Constructor argument:

- `_escrowContract`: paste the BugBountyEscrow address from Step 4

Click **Deploy**

Copy the BugBountyRegistry deployed address



Step 4:- Paste the BugBountyEscrow address to mint BBT tokens then click the transact button

- Expand the deployed BugBountyToken contract in the left panel
- Find the setMinter function
- Paste the BugBountyEscrow address
- Click **transact**

DEPLOY & RUN TRANSACTIONS

ENVIRONMENT: Remix VM (Cancun)

ACCOUNT: 0x583...eddC4 (99.999999999)

GAS LIMIT: Estimated Gas

VALUE: 0 Wei

CONTRACT: BugBountyRegistry - BugBounty/Bug

Deploy address_escrowContract

At Address Load contract from Address

Transactions recorded 1

```

22 contract BugBountyRegistry {
23
24     // -- Types
25
26     enum Severity { None, Informational, Low, Medium, High, Critical }
27
28     struct HunterProfile {
29         uint256 totalSubmissions;
30         uint256 totalApprovals;
31         uint256 totalEarnedWei;
32         uint256 reputationScore; // simple formula: approvals * 10 + totalEarned / 1e15
33     }
34
35     struct BountyRecord {
36         uint256 bountyId;
37         address sponsor;
38         Severity severity; // set by sponsor on creation or after approval

```

creation of BugBountyRegistry pending...

[vm] from: 0x583...eddC4 to: BugBountyRegistry.(constructor) value: 0 wei data: 0x608...33fa8 logs: 0 hash: 0x168...784f3

creation of BugBountyRegistry pending...

[vm] from: 0x583...eddC4 to: BugBountyRegistry.(constructor) value: 0 wei data: 0x608...33fa8 logs: 0 hash: 0x82a...6fd18

Step 5:- Continuing from the step 4

DEPLOYED CONTRACTS

BUGBOUNTYTOKEN AT 0XD91

Balance: 0 ETH

approve address spender, uint256 amount

mint address to, uint256 amount

setMinter 0xd8b934580c35a11858c

transfer address to, uint256 amount

transferFrom address from, address to, uint256 amount

```

22 contract BugBountyRegistry {
23
24     // -- Types
25
26     enum Severity { None, Informational, Low, Medium, High, Critical }
27
28     struct HunterProfile {
29         uint256 totalSubmissions;
30         uint256 totalApprovals;
31         uint256 totalEarnedWei;
32         uint256 reputationScore; // simple formula: approvals * 10 + totalEarned / 1e15
33     }
34
35     struct BountyRecord {
36         uint256 bountyId;
37         address sponsor;
38         Severity severity; // set by sponsor on creation or after approval

```

[vm] from: 0x583...eddC4 to: BugBountyRegistry.(constructor) value: 0 wei data: 0x608...33fa8 logs: 0 hash: 0x168...784f3

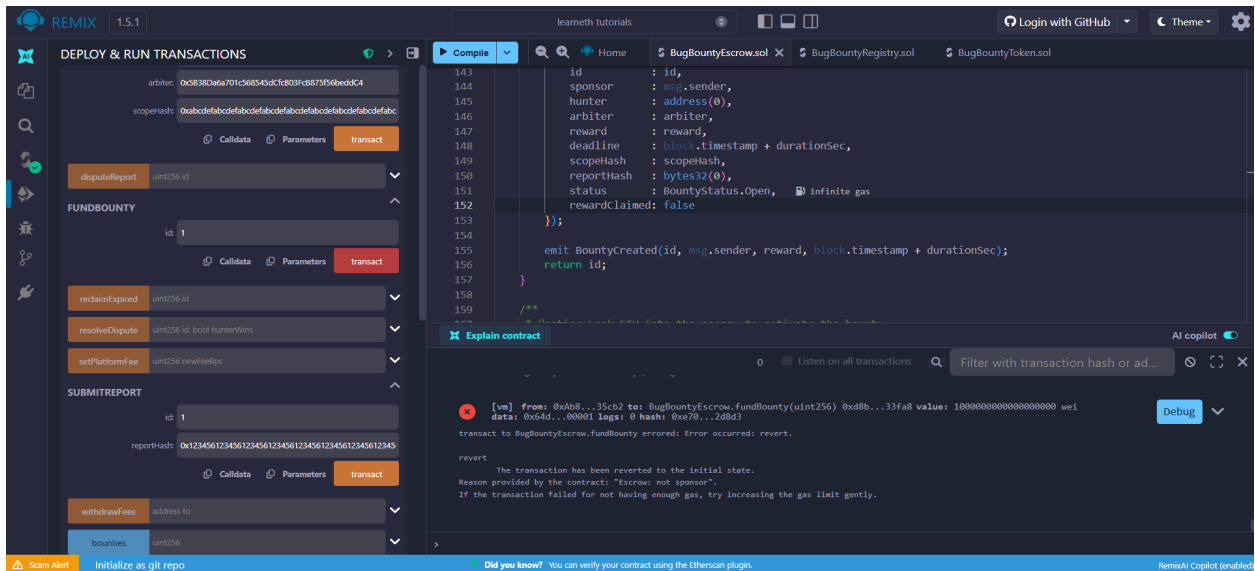
creation of BugBountyRegistry pending...

[vm] from: 0x583...eddC4 to: BugBountyRegistry.(constructor) value: 0 wei data: 0x608...33fa8 logs: 0 hash: 0x82a...6fd18

transact to BugBountyToken.setMinter pending ...

[vm] from: 0x583...eddC4 to: BugBountyToken.setMinter(address) 0xd91...39138 value: 0 wei data: 0xfca...33fa8 logs: 1 hash: 0xe95...a1cc8

Step 6:- Call fundBountywith id = 1 and set Value to 1 ETH



Step 7 :- You need 3 fake people (accounts). At the **top of the Deploy panel** in Remix, there's a dropdown that says **"Account"** Click it — you'll see **10 fake accounts**, each has **100 ETH** (free test money!)

Step 8:- Sponsor creates a bounty

Switch to Account 2 (the Sponsor) from the Account dropdown. Now go to your deployed **BugBountyEscrow** contract and click **createBounty**

Fill in that exactly like this

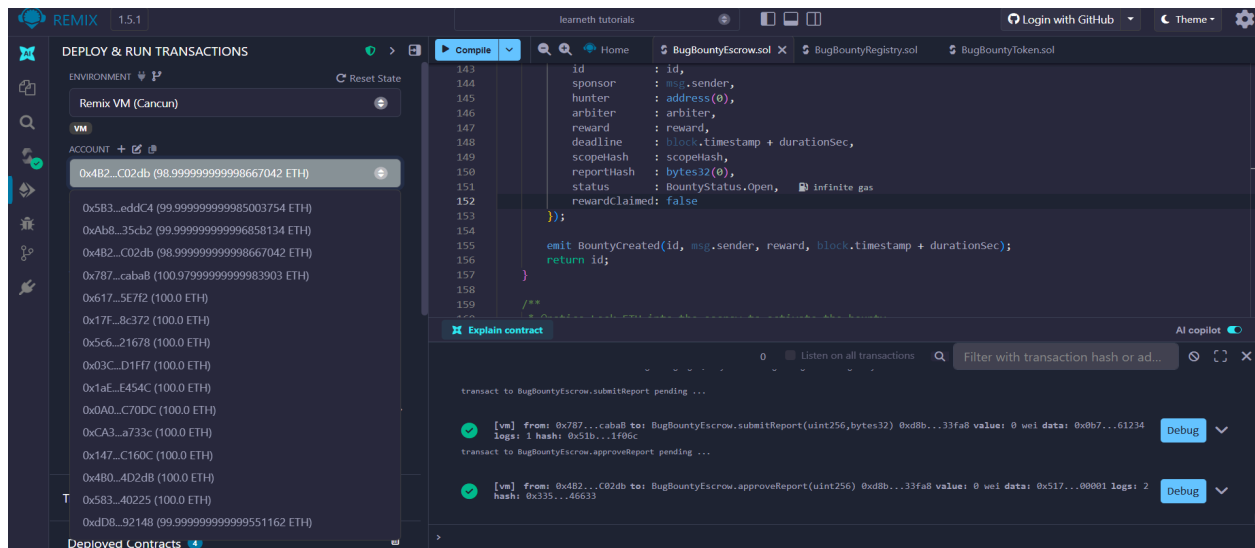
[illegible]

durationSec $\rightarrow$ 2592000

arbiter → [paste Account 1 address]

scopeHash $\rightarrow$ 0xabcd....

Step 9:- Register bounty in the registry, then there are 3 people in the accounts part there are 10 fake accounts, then one owner at the top, then there is sponsor, the sponsor will create bounty, sponsor then locks the bounty, hunter submits their bug report and the sponsor approves and then the hunter gets paid, **then the value of that hunter raises**



Conclusion

The Decentralized Bug Bounty Escrow System demonstrates how blockchain technology can solve real trust problems in the cybersecurity industry. By replacing middlemen with smart contracts, it ensures that hunters always get paid fairly and sponsors only release funds when satisfied all enforced automatically by code, not promises.

With three interconnected contracts handling payments, reputation, and dispute resolution, this system creates a transparent and tamper-proof environment where security researchers can work with confidence. The addition of BBT reward tokens further incentivizes hunters by building their on-chain reputation with every successful submission.

This project proves that decentralized systems are not just a concept they are practical, deployable tools that can reshape how industries handle trust, payments, and accountability. As Web3 continues to grow, platforms like this represent the future of fair and transparent collaboration between companies and the security community.